John Chiero

Professor Kanemoto

CPSC-39

22 April 2026

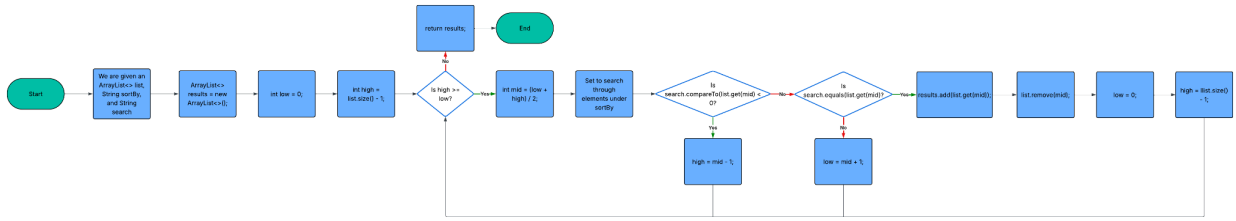
Library Manager - Final Project Report

Program Overview:

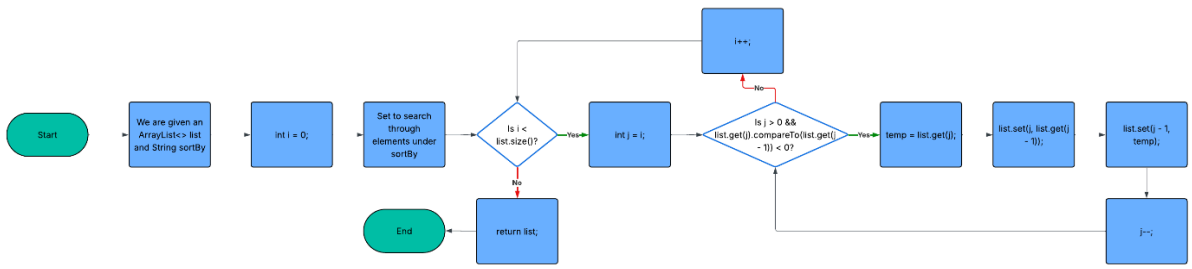
This program allows the user to use provided tools to manage library resources. To run the program, simply click the run button and follow the prompts given. This usually consists of inputting a number correlated with the desired feature or typing needed information to complete the process. The features of this program consists of the following: search for a book, get a list of all books, search for a member, get a list of all members, check out a book to a member, allow a member to check in a book, let a member reserve/queue for a book, add and remove a book, and add and remove a member. All of these tools allow users to quickly and efficiently access information from the library, as well as add or remove information when needed.

Algorithms:

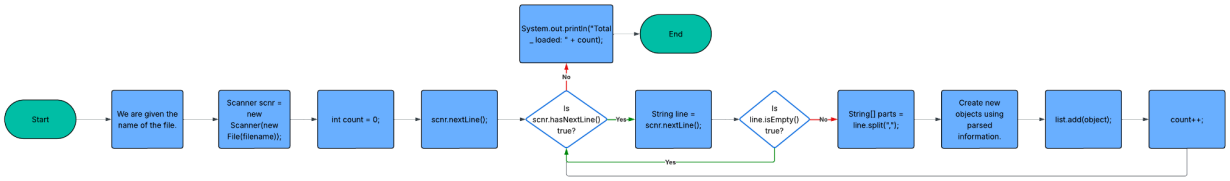
- Binary Search: My program consists of two binary search algorithms, one for searching through the list of books and the other for the members. These algorithms allow a book or member to be searched for and found if they are in the system. Otherwise, the user is notified that there is no such book/member in the system currently. The Big-O complexity of this algorithm is O(log n) for the worst case and O(1) for the best case.



- Insertion Sort: My program consists of two insertion sort algorithms, just like binary search, one for sorting through the list of books and the other for the members. These algorithms will sort the list of books/members, based on a variable sortBy, from least to greatest and/or in alphabetical order. The Big-O complexity of this algorithm is $O(n^2)$ for the worst case and O(n) for the best case.



- Loading algorithm: My program consists of three loading algorithms. These are used to load the information found in text files, Books.txt, Members.txt, and Scenes.txt, and put it into corresponding ArrayList. This makes the information easily accessible for many uses throughout the program. The Big-O complexity of this algorithm is O(n) for both the worst and best case.



Data Structures:

- Arrays: Mainly used for temporary information, such as gathering information from a file to put into a larger list. Arrays were chosen to preserve memory, since they were usually used for smaller functions of the program that are meant to be very quick.

- ArrayLists: Mainly used to store lists of all the books, members, and scenes that are used in the program. ArrayLists were chosen to provide more flexibility, which is helpful when storing information used throughout the whole program.
- Queue: Used to store the queue for a book. Since I was making a queue list for members to be able to claim their turn with a book that they are interested in, it only made sense to use the queue data structure that has built in functions perfect for this system.

Development Reflection:

One problem I encountered with my program was that when I used my binary search algorithm to find an element a part of ArrayList<Book> books, I would find the element, but it would no longer be a part of the ArrayList. At first, I thought it was an issue with how I used binary search and somehow removed the element. However, I couldn't find anything that indicated removing anything from books. So, I decided to look into how I created my binary search algorithm. I found that I was removing found elements from the original ArrayList instead of a copy, so I simply added a variable to store a copy of the list to work on and that solved my issue.

One design improvement I discovered was to use a text file to store the different scenes that would be used for the Library Manager. Originally, I was thinking of just having it all in the code. However, I thought that would be kind of messy and that is when I had the idea to do a similar thing we did in the Escape Room project. I created a text file with various scenes, each with their own scene IDs to be able to navigate among them. This reduces the length of my code and keeps it in a cleaner, easier to use format.

Something I would look into adding in a version two of my Library Manager is a way to save information. How I think I could go about that is to update the text files with altered

information from the user's use of the program, so the next time they boot up the program that information would be loaded instead of what I currently have in place for testing purposes.